

Using Mandelbrot Set and N-Body Simulation as Pedagogic Tools in the Teaching of Parallel Computing Concepts

Rayan Raddatz de Matos, Francisco Pegoraro Etcheverria, Kenichi Brumati, Enzo Lisboa Peixoto, Pedro Henrique Boniatti Colle, Otho Jose Sirtoli Marcondes, Lucas Mello Schnorr

{rayan.raddatz, fpetcheverria, kbrumati, elpeixoto, phbcolle, ojsmarcondes, schnorr}@inf.ufrgs.br

CARLA 2026

Motivation

The gap in PDC teaching

- IEEE/ACM guidelines have listed PDC as required knowledge since 2013.

The gap in PDC teaching

- IEEE/ACM guidelines have listed PDC as required knowledge since 2013.
- In practice, decomposition, granularity, load balance, speedup and scalability are still taught in late-stage electives.

The gap in PDC teaching

- IEEE/ACM guidelines have listed PDC as required knowledge since 2013.
- In practice, decomposition, granularity, load balance, speedup and scalability are still taught in late-stage electives.
- Students arriving from high school have no intuition for parallelism at all.

The gap in PDC teaching

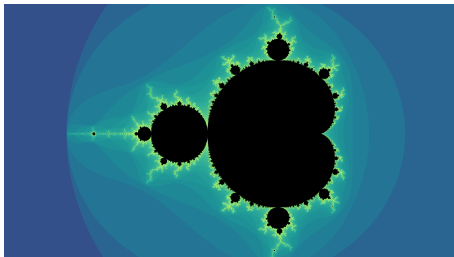
- IEEE/ACM guidelines have listed PDC as required knowledge since 2013.
- In practice, decomposition, granularity, load balance, speedup and scalability are still taught in late-stage electives.
- Students arriving from high school have no intuition for parallelism at all.
- Most existing tools visualize code the student wrote, so they assume a programming background a general audience does not have.

Our proposal

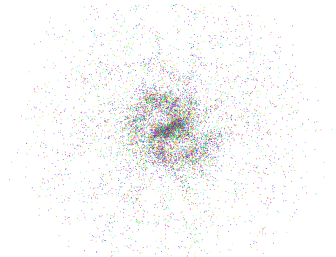
Two open-source interactive applications for teaching parallel computing concepts through direct observation, with no programming prerequisite.

Our proposal

Two open-source interactive applications for teaching parallel computing concepts through direct observation, with no programming prerequisite.



Mandelbrot: embarrassingly parallel



N-Body: tightly coupled

Methods and Materials

Common client/server design

- **Client:** graphical application on a local machine, handling user input and rendering.

Common client/server design

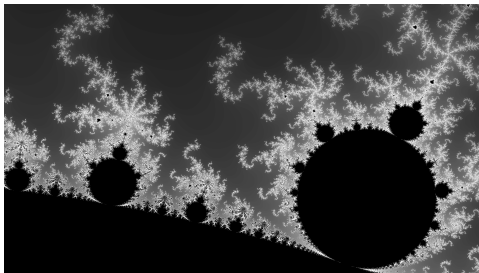
- **Client:** graphical application on a local machine, handling user input and rendering.
- **Server:** MPI application on a cluster, doing the domain decomposition and the numerical work.

Common client/server design

- **Client:** graphical application on a local machine, handling user input and rendering.
- **Server:** MPI application on a cluster, doing the domain decomposition and the numerical work.
- They communicate over TCP, and results stream back block by block.

Common client/server design

- **Client:** graphical application on a local machine, handling user input and rendering.
- **Server:** MPI application on a cluster, doing the domain decomposition and the numerical work.
- They communicate over TCP, and results stream back block by block.
- The user sees the interaction dispatch work to a larger system.

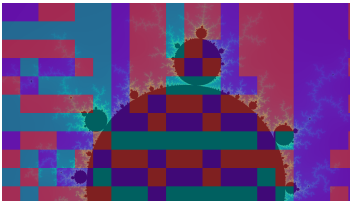


Mandelbrot: decomposition and granularity

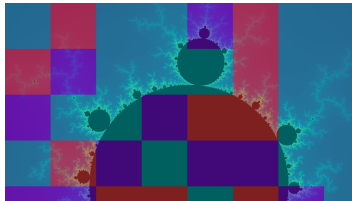
Rank 0 splits the canvas into square blocks whose side is the granularity chosen by the user, and the workers request blocks on demand.



Granularity 10



Granularity 100



Granularity 250

Mandelbrot: load balance

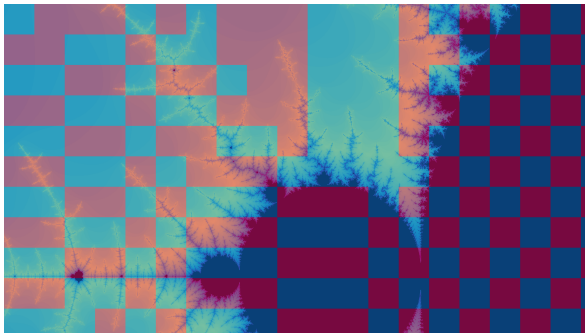
- The imbalance is inherent to the problem: blocks of equal pixel size carry very unequal cost.

Mandelbrot: load balance

- The imbalance is inherent to the problem: blocks of equal pixel size carry very unequal cost.
- The demand-driven coordinator handles it dynamically, since whoever finishes first asks for the next block.

Mandelbrot: load balance

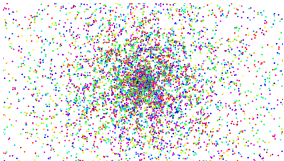
- The imbalance is inherent to the problem: blocks of equal pixel size carry very unequal cost.
- The demand-driven coordinator handles it dynamically, since whoever finishes first asks for the next block.
- The vertical stripes are a single worker taking the cheap blocks above a costly boundary block.



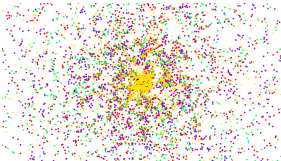
N-Body simulation

$$\mathbf{a}_i = \sum_{j \neq i} \frac{G m_j \mathbf{r}_{ij}}{(\|\mathbf{r}_{ij}\|^2 + \epsilon^2)^{3/2}}$$

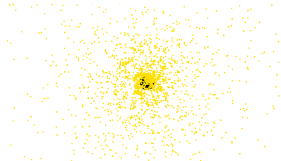
- $O(N^2)$ per step, with the particles split into equal contiguous slices, one per rank.
- Gravity is long-range, so every iteration ends in a collective exchange of all positions.
- Load balance is even by construction, and the limiting factor is communication.



Random coloring



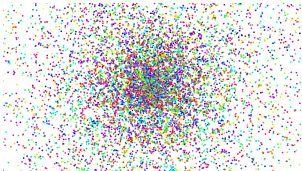
Per-worker coloring



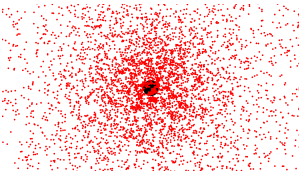
Rank 3 only

N-Body: speedup and scalability

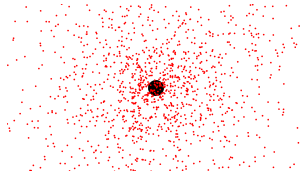
With more workers each slice gets smaller, so there is less to compute before the next iteration and the simulation runs visibly faster. The position exchange after every step is what limits how far this scales.



15000 particles



Rank 0, 2 workers



Rank 0, 8 workers

Concept mapping

Concept	Mandelbrot	N-Body
Problem coupling	Embarrassingly parallel	Tightly coupled
Domain decomposition	Square blocks, variable size	Equal contiguous particle slices
Granularity	Explicit user-controlled parameter	Fixed (one slice per rank)
Load balance	Irregular, dynamically managed	Balanced by construction
Speedup & scalability	Granularity-dependent	Communication-bounded

We chose the two applications to be complementary. Where one illustrates a concept directly, the other offers the contrasting case.

Results and Discussion

Pedagogical deployment: UFRGS Open Day

Both editions received waves of prospective students, mostly high schoolers with no computing background. We used Mandelbrot in 2025 and N-Body in 2026, with the client on a local workstation and the server on 6 PCAD nodes, 144 cores in total.

Pedagogical deployment: UFRGS Open Day

Both editions received waves of prospective students, mostly high schoolers with no computing background. We used Mandelbrot in 2025 and N-Body in 2026, with the client on a local workstation and the server on 6 PCAD nodes, 144 cores in total.

In the first stage, visitors interacted with the application while we walked them through decomposition, granularity, load distribution and the effect of adding workers. The per-worker coloring was the most effective element here.

Pedagogical deployment: UFRGS Open Day

Both editions received waves of prospective students, mostly high schoolers with no computing background. We used Mandelbrot in 2025 and N-Body in 2026, with the client on a local workstation and the server on 6 PCAD nodes, 144 cores in total.

In the first stage, visitors interacted with the application while we walked them through decomposition, granularity, load distribution and the effect of adding workers. The per-worker coloring was the most effective element here.

In the second stage, we took them to see the machines that had run their computation. Connecting the image on screen to the physical hardware was the highlight of the visit.

What worked, and what did not

- The most effective sequence was to run the application locally with few workers, then switch to the full cluster.

What worked, and what did not

- The most effective sequence was to run the application locally with few workers, then switch to the full cluster.
- Mandelbrot also worked in formal settings: an undergraduate Operating Systems course, and a full factorial performance study published separately.

What worked, and what did not

- The most effective sequence was to run the application locally with few workers, then switch to the full cluster.
- Mandelbrot also worked in formal settings: an undergraduate Operating Systems course, and a full factorial performance study published separately.
- On the limitations, the sessions are short and informal, which rules out any formal assessment, and the cluster tour is not reproducible everywhere.

What worked, and what did not

- The most effective sequence was to run the application locally with few workers, then switch to the full cluster.
- Mandelbrot also worked in formal settings: an undergraduate Operating Systems course, and a full factorial performance study published separately.
- On the limitations, the sessions are short and informal, which rules out any formal assessment, and the cluster tour is not reproducible everywhere.
- With fewer parameters to explore, N-Body visitors ended up following the guided explanation more closely than Mandelbrot ones.

Conclusion

Conclusion and future work

- Two open-source MPI applications that make decomposition, granularity, load balance, speedup and scalability directly observable.

Conclusion and future work

- Two open-source MPI applications that make decomposition, granularity, load balance, speedup and scalability directly observable.
- They serve as an entry point for audiences with no computing background, and still carry enough depth for formal academic use.

Conclusion and future work

- Two open-source MPI applications that make decomposition, granularity, load balance, speedup and scalability directly observable.
- They serve as an entry point for audiences with no computing background, and still carry enough depth for formal academic use.
- As future work, we intend to assess learning outcomes, cover additional concepts, reach more courses, and convey the balance between network and compute capability.

<https://github.com/rddtz/pcad-applications>

Questions?

Contact: rayan.raddatz@inf.ufrgs.br

<https://github.com/rddtz/pcad-applications>

